



Università degli Studi di Bergamo

Dipartimento di Ingegneria

Corso di Laurea in Ingegneria Informatica

Anno Accademico 2025/2026

DESIGN

Ingegneria del Software

The Scouting Suite

Brivio Andrea — Matricola: 1089359

Fischetti Alessandro — Matricola: 1080491

Pesenti Lorenzo — Matricola: 1072635

Indice

1	Introduzione	2
1.1	Scopo del Documento	2
1.2	Contesto del Progetto	2
1.3	Glossario	2
2	Modellazione UML	3
2.1	Use Case Diagram	3
2.2	Class Diagram	3
2.3	Package Diagram	3
2.4	Component Diagram	4
2.5	Activity Diagram	4
2.6	Sequence Diagrams	4
2.7	Communication Diagram	4
2.8	State Machine Diagram	4
3	Architettura del Software	5
3.1	Stile Architetturale	5
3.2	Pattern Architetturali	5
3.2.1	Layered Architecture	5
3.2.2	MVC (Model-View-Controller)	5
3.2.3	Dependency Injection	5
3.3	Componenti e Connettori	5
3.4	Tecnologie e Librerie	6
4	Design del Software	6
4.1	Design dei Package	6
4.2	Design Pattern Applicati	7
4.2.1	Factory Method Pattern	7
4.2.2	Strategy Pattern	7
4.2.3	Specification Pattern	7
4.2.4	Repository Pattern	7
4.2.5	DTO Pattern	8
4.2.6	Component Pattern	8
4.3	Design dell'Entità Principale: Player	8
4.4	Design del Sistema di Filtri	9
4.5	Principi SOLID Applicati	10
4.6	Metriche di Qualità del Design	10
4.7	Considerazioni sul Design	10
5	Conclusioni	11
5.1	Risultati del Design	11
5.2	Tracciabilità Requisiti-Design	11
5.3	Limitazioni e Sviluppi Futuri	12
5.4	Valutazione Complessiva	12

1 Introduzione

1.1 Scopo del Documento

Questo documento descrive in dettaglio le scelte architetture, progettuali e implementative del software *The Scouting Suite*. Seguendo le linee guida del corso di Ingegneria del Software e lo standard IEEE 1016, il documento fornisce:

- Una panoramica completa dell'architettura a layer
- I diagrammi UML prodotti con Papyrus/Eclipse
- I pattern di design applicati e la loro giustificazione
- Le metriche di qualità del codice
- Le tecnologie e framework utilizzati

1.2 Contesto del Progetto

The Scouting Suite è una piattaforma completa per lo scouting calcistico professionistico che consente:

- L'analisi avanzata di giocatori tramite **100+ metriche statistiche** (xG, xAG, passes completati, tackles, ecc.)
- Il filtraggio dinamico e combinabile di dati complessi
- La visualizzazione intuitiva tramite interfaccia web interattiva
- L'importazione/export di dataset da fonti esterne (CSV)
- La gestione di database relazionali per dati storici

1.3 Glossario

- **ETL (Extract-Transform-Load)**: Processo di estrazione, trasformazione e caricamento dati
- **JPA (Java Persistence API)**: Specifica Java per il mapping oggetti-relazionale
- **Specification Pattern**: Pattern per costruire query dinamiche
- **Anemic Domain Model**: Pattern dove le entità contengono solo dati, non logica
- **Dependency Injection**: Pattern per la gestione delle dipendenze
- **DTO (Data Transfer Object)**: Oggetto per trasferire dati tra layer
- **Spring Boot**: Framework per applicazioni enterprise Java
- **Vaadin**: Framework per interfacce web in Java

2 Modellazione UML

Tutti i diagrammi UML sono stati prodotti utilizzando **Papyrus for Eclipse** e sono disponibili nella cartella `Docs/DiagramUML/bin/`. Ogni diagramma è composto dai file `.uml`, `.notation`, `.di` e `.aird`.

2.1 Use Case Diagram

Attori identificati:

- **Scout/Analista:** Utente professionale che cerca giocatori per talent scouting
- **Utente Generico:** Visualizza statistiche e confronta giocatori

Casi d'uso principali:

- Gestione dataset giocatori (importazione/export)
- Applicazione filtri avanzati (età, squadra, statistiche)
- Visualizzazione personalizzata delle metriche
- Analisi comparativa tra giocatori

2.2 Class Diagram

Principali classi modellate:

- **Player:** Entità JPA con 100+ attributi statistici
- **PlayerRepository:** Interfaccia Spring Data JPA
- **ScoutingService:** Servizio core di business logic
- **PlayerSpecificationFactory:** Factory per query dinamiche
- **MainView:** Interfaccia utente Vaadin
- **DataProcessingService:** Servizio ETL per caricamento CSV

2.3 Package Diagram

Struttura a package:

- `com.scouting:` Package root
- `com.scouting.data:` Layer di persistenza
 - `model:` Entità JPA (Player)
 - `repository:` Interfacce Spring Data
- `com.scouting.service:` Layer di business logic
 - `specification:` Pattern Specification
- `com.scouting.ui:` Layer di presentazione
- `com.scouting.config:` Classi di configurazione

2.4 Component Diagram

Componenti principali:

- **Presentation Component:** Interfaccia Vaadin (MainView)
- **Business Component:** Servizi applicativi (ScoutingService)
- **Persistence Component:** Layer dati (PlayerRepository)
- **ETL Component:** Processore dati (DataProcessingService)

2.5 Activity Diagram

Flussi principali modellati:

- Avvio applicazione e caricamento dati
- Configurazione filtri da parte dell'utente
- Costruzione ed esecuzione query dinamiche
- Visualizzazione risultati

2.6 Sequence Diagrams

Sequenze critiche documentate:

- Inizializzazione Spring Boot e caricamento automatico dati
- Interazione utente → controller → service → repository
- Costruzione query tramite Specification Pattern

2.7 Communication Diagram

Interazioni modellate:

- Collaborazione tra MainView, ScoutingController, ScoutingService
- Comunicazione con PlayerSpecificationFactory per query dinamiche
- Interazione con il database tramite PlayerRepository

2.8 State Machine Diagram

Stati principali:

- **Initialization:** Caricamento dati e setup
- **Idle:** In attesa di input utente
- **Filtering:** Applicazione criteri di ricerca
- **Processing:** Elaborazione query e risultati
- **Display:** Visualizzazione output

3 Architettura del Software

3.1 Stile Architeturale

Il progetto adotta un'architettura a **layer (3-tier)** con chiara separazione delle responsabilità:

Layer	Responsabilità	Tecnologie
Presentation	Interfaccia utente e controller	Vaadin, Spring MVC
Business	Logica applicativa e servizi	Spring Services, Factory Pattern
Persistence	Accesso e gestione dati	Spring Data JPA, H2 Database

3.2 Pattern Architeturali

3.2.1 Layered Architecture

Separazione in tre layer principali per garantire:

- **Modularità:** Ogni layer ha responsabilità ben definite
- **Manutenibilità:** Modifiche in un layer non impattano gli altri
- **Testabilità:** Ogni layer testabile in isolamento

3.2.2 MVC (Model-View-Controller)

Implementato attraverso:

- **Model:** Entità JPA (`Player`) e servizi (`ScoutingService`)
- **View:** Componenti Vaadin (`MainView`)
- **Controller:** `ScoutingController` che orchestra le interazioni

3.2.3 Dependency Injection

Gestito nativamente da Spring Boot:

- Iniezione di dipendenze via costruttore
- Configuration tramite annotazioni `@Autowired`
- Scopi: Singleton, Prototype, Request, Session

3.3 Componenti e Connettori

Componenti principali:

- **Application:** Entry point Spring Boot (`@SpringBootApplication`)
- **BrowserLauncher:** Componente per UX enhancement (`@Component`)
- **DataProcessingService:** Pipeline ETL per dati CSV (`@Service`)
- **ScoutingService:** Core business logic (`@Service`)

- **PlayerSpecificationFactory**: Factory per query dinamiche (@Component)
- **PlayerRepository**: Astrazione dati Spring Data JPA (@Repository)
- **MainView**: Interfaccia principale Vaadin (@Route)
- **ScoutingController**: Controller MVC (@Component)

Connettori:

- **Chiamate di metodo**: Tra componenti nello stesso layer
- **Dependency Injection**: Tra layer diversi via Spring
- **JPA/Hibernate**: Per persistenza su database
- **HTTP/WebSocket**: Comunicazione client-server (Vaadin)

3.4 Tecnologie e Librerie

Categoria	Tecnologia	Scopo
Framework	Spring Boot 3.x	Container IoC, gestione ciclo vita
UI Framework	Vaadin 24	Interfaccia web type-safe in Java
Persistence	Spring Data JPA + H2	ORM e database in-memory
Build Tool	Apache Maven	Gestione dipendenze e lifecycle
Testing	JUnit 5, Mockito	Unit e integration test
Logging	Log4j 2	Tracciamento e debug
Analisi Codice	SonarLint	Qualità e sicurezza codice
UML Modeling	Papyrus for Eclipse	Diagrammi UML standard

4 Design del Software

4.1 Design dei Package

La struttura del progetto segue le convenzioni Spring Boot:

```
src/main/java/com/scouting/
  Application.java           # Entry point
  config/
    BrowserLauncher.java    # Auto-browser launch
  data/
    model/
      Player.java           # JPA Entity (100+ attributi)
    repository/
      PlayerRepository.java  # Spring Data JPA
  service/
    specification/
      PlayerSpecificationFactory.java
    strategy/
      StatQueryStrategy.java
  DataProcessingService.java # ETL pipeline
```

ScoutingService.java	# Business logic
PlayerFilterRequest.java	# DTO (record)
StatFilterCriteria.java	# Filter criteria
ui/	
MainView.java	# Vaadin main view
ScoutingController.java	# MVC Controller
FilterRowComponent.java	# Custom UI component

4.2 Design Pattern Applicati

4.2.1 Factory Method Pattern

Implementazione: PlayerSpecificationFactory

- **Scopo:** Creare oggetti Specification<Player> in modo dinamico
- **Motivazione:** Separare la costruzione di query complesse dalla loro esecuzione
- **Vantaggi:** Estendibile (Open/Closed Principle), testabile

4.2.2 Strategy Pattern

Implementazione: StatQueryStrategy e implementazioni concrete

- **Scopo:** Algoritmi intercambiabili per diversi tipi di filtro statistico
- **Motivazione:** Gestire variazioni nei criteri di filtro (interi, decimali, stringhe)
- **Vantaggi:** Elimina switch-case complessi, facilita l'aggiunta di nuovi tipi

4.2.3 Specification Pattern

Implementazione: Spring Data JPA Specification<T>

- **Scopo:** Costruire query dinamiche con criteri combinabili
- **Motivazione:** Supportare filtri complessi AND/OR senza query fisse
- **Vantaggi:** Query type-safe, riutilizzabili, composabili

4.2.4 Repository Pattern

Implementazione: PlayerRepository extends JpaRepository<Player, Long>

- **Scopo:** Astrazione dell'accesso ai dati
- **Motivazione:** Separare logica business dalla persistenza
- **Vantaggi:** Cambiare database senza modificare business logic

4.2.5 DTO Pattern

Implementazione: PlayerFilterRequest (Java Record)

- **Scopo:** Trasferire dati tra layer (UI → Service)
- **Motivazione:** Evitare troppi parametri nei metodi, incapsulare dati correlati
- **Vantaggi:** Immutabile, conciso, self-documenting

4.2.6 Component Pattern

Implementazione: FilterRowComponent, BrowserLauncher

- **Scopo:** Componenti riutilizzabili e autonomi
- **Motivazione:** Incapsulare logica UI complessa in unità gestibili
- **Vantaggi:** Riutilizzo, testabilità, separazione concern

4.3 Design dell'Entità Principale: Player

```
@Entity
@Table(name = "player")
public class Player {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private Integer age;
    private String position;
    private String squad;
    private String competition;

    // Playing Time
    private Integer matches;
    private Integer starts;
    private Integer minutes;
    @Column(name = "nineties")
    private Double ninetyS;

    // Standard Stats
    private Integer goals;
    private Integer assists;
    @Column(name = "g_plus_a")
    private Integer gPlusA;

    // Expected Metrics
    private Double xg;
    private Double npvg;
```

```

private Double xag;
@Column(name = "npxg_plus_xag")
private Double npxgPlusXag;

// ... 80+ altri attributi statistici

// Costruttore vuoto (JPA requirement)
public Player() {}

// Getter e setter per tutti gli attributi
// (Anemic Domain Model Pattern)
}

```

Scelte di design per l'entità Player:

- **Anemic Domain Model:** L'entità contiene solo dati, la logica è nei servizi
- **Naming columns:** Mappatura esplicita tra nomi Java e colonne database
- **Separazione categorie:** Attributi raggruppati per area (Playing Time, Expected Metrics, ecc.)
- **Immutabilità:** Solo getter/setter, nessuna logica di business nell'entità

4.4 Design del Sistema di Filtri

Componenti del sistema di filtri:

1. **MainView:** Raccoglie input utente da vari componenti UI
2. **PlayerFilterRequest:** Incapsula tutti i criteri in un DTO
3. **ScoutingService:** Riceve la richiesta e delega al factory
4. **PlayerSpecificationFactory:** Costruisce la Specification usando Strategy Pattern
5. **PlayerRepository:** Esegue la query e restituisce risultati

Flusso dati:

```

UI Input → PlayerFilterRequest → ScoutingService →
PlayerSpecificationFactory → Specification<Player> →
PlayerRepository → List<Player> → UI Display

```

4.5 Principi SOLID Applicati

Principio	Applicazione nel Progetto	Beneficio
Single Responsibility	Ogni classe ha una responsabilità chiara (es: PlayerSpecificationFactory crea solo Specification)	Manutenzione più semplice, testing isolato
Open/Closed	Il sistema di filtri è estendibile (nuovi tipi di filtro) senza modificare codice esistente	Estensibilità senza rischi di regressione
Liskov Substitution	Le strategie di filtro sono intercambiabili senza rompere il sistema	Flessibilità e riutilizzo
Interface Segregation	Interfacce piccole e specifiche (es: StatQueryStrategy)	Dipendenze minimali, accoppiamento ridotto
Dependency Inversion	Dipendenze verso astrazioni, non implementazioni (Spring DI)	Testabilità, modularità

4.6 Metriche di Qualità del Design

Metrica	Valore Target	Strumento
Complessità Ciclomantica	< 10 per metodo	SonarLint, Checkstyle
Accoppiamento Afferente (Ca)	Basso (dipendenze in entrata)	JDepend
Accoppiamento Efferente (Ce)	Moderato (dipendenze in uscita)	JDepend
Numero di Classi per Package	5-15	Analisi struttura
Depth of Inheritance Tree	< 3	Analisi gerarchia
Commenti per Linea di Codice	20-30%	Javadoc coverage
Test Coverage	> 80%	JaCoCo
Duplicazione Codice	< 5%	SonarLint
Violazioni Regole	0 critiche, < 10 maggiori	SonarQube/SonarLint

4.7 Considerazioni sul Design

Punti di forza:

- **Separazione netta dei concern:** UI, business, persistenza chiaramente separati
- **Estensibilità:** Pattern Factory e Strategy facilitano nuove feature
- **Testabilità:** Dependency injection permette mocking e unit test isolati
- **Manutenibilità:** Codice ben organizzato, documentato, con naming significativo
- **Performance:** Query ottimizzate tramite JPA Specifications, caching Spring

Scelte progettuali significative:

1. **Anemic Domain Model:** Scelto per separazione netta dati/logica
2. **Vaadin vs React/Angular:** Scelto per coerenza tech stack (solo Java)
3. **H2 in-memory:** Per sviluppo/test, ma architettura pronta per PostgreSQL
4. **Java Records:** Per DTO immutabili (Java 17+ feature)
5. **ETL all'avvio:** Caricamento dati all'avvio per UX immediata

Decisioni architetturali:

- **Monolith vs Microservices:** Architettura monolitica per semplicità iniziale
- **REST API vs Vaadin Direct:** Vaadin per UI integrata, ma API REST estendibili
- **Database Schema:** Mappatura 1:1 con CSV sorgente per semplicità ETL

5 Conclusioni

5.1 Risultati del Design

Il design presentato per *The Scouting Suite* rappresenta una soluzione robusta, mantenibile ed estensibile per l'analisi di dati calcistici. Le principali realizzazioni includono:

- **Architettura ben strutturata:** Separazione chiara in layer con responsabilità definite
- **Pattern appropriati:** Applicazione di Factory, Strategy, Specification per gestire complessità
- **Modellazione completa:** Set completo di diagrammi UML che guidano lo sviluppo
- **Qualità del codice:** Metriche di qualità applicate tramite strumenti automatizzati
- **Testabilità:** Design che facilita testing automatico e manuale

5.2 Tracciabilità Requisiti-Design

Tutte le scelte di design sono tracciabili ai requisiti identificati:

- Filtri avanzati → Specification Pattern + Strategy Pattern
- Performance query → Spring Data JPA + H2 optimization
- UX immediata → ETL all'avvio + BrowserLauncher
- Manutenibilità → Layered Architecture + SOLID principles
- Estensibilità → Factory Pattern + Open/Closed Principle

5.3 Limitazioni e Sviluppi Futuri

- **Limitazioni attuali:**

- Database in-memory (H2) non persistente tra riavvii
- Assenza di autenticazione/autorizzazione
- UI non completamente responsive per mobile

- **Sviluppi futuri pianificati:**

- Migrazione a database persistente (PostgreSQL)
- Implementazione autenticazione OAuth2
- API REST per integrazione con sistemi esterni
- Dashboard analitiche avanzate
- Machine learning per talent prediction

5.4 Valutazione Complessiva

Il design presentato dimostra l'applicazione corretta dei principi di Ingegneria del Software:

- **Coerenza:** Tra modelli UML e implementazione Java
- **Documentazione:** Completa e tracciabile
- **Qualità:** Metriche oggettive e strumenti di analisi
- **Manutenibilità:** Struttura chiara e pattern appropriati
- **Testabilità:** Facilitata da dependency injection e separazione concern

Il documento costituisce quindi una base solida per lo sviluppo, la manutenzione e l'evoluzione futura di *The Scouting Suite*.